# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 1 – Industry Problem Solving Task

| | | | |
|---|---|---|---|
| **Course Code:** | CSA0612 | **Course Title:** | Design and Analysis of Algorithms |
| **CO Assessed:** | **CO4:** Articulation (BL4, BL5) | **Assessment:** | Assessment Tool 1 –Coding Challenge Task |
| **Weightage:** | 20% of CIA | **Total Marks:** | 100 |
| **CO4 Statement:** | Solve optimization problems using dynamic programming and greedy strategies. | | |
| **Student Name:** | JAYAKRISHNA.V | **Reg. No.:** | 192421259 |
| **Section / Batch:** | 1 | **Date:** | 27.08.2026 |

**Instructions to Students**

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

| Question Type | Focus Area | Questions |
|---|---|---|
| **Coding Challenge Task** | Focus on Code and analyze optimization problems using dynamic programming and greedy strategies | Q1 |

## SECTION A – Industry Problem Solving Task

**Code of Conduct**

*I JAYAKRISHNA.V(192421259) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: JAYAKRISHNA.V_____

## RUBRICS FOR CALCULATION

### Industry Problem Solving Task

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Industry Problem | 20% | Comprehensive understanding of real-world problem and constraints | Good understanding with minor gaps in requirements | Basic understanding; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25% | Applies advanced and relevant concepts effectively to solve the problem | Applies appropriate concepts with minor errors | Limited application of concepts | Incorrect or no application of concepts |
| Solution Design | 25% | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps | Basic solution with limited feasibility | Unclear or impractical solution |
| Use of Tools | 15% | Effective use of modern tools, technologies, and real data | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools | Minimal or incorrect use of tools |
| Analysis | 10% | Thorough analysis with validated results and performance evaluation | Good analysis with reasonable validation | Basic analysis with limited validation | Poor or no analysis |
| Professionalism | 5% | Highly professional, well-documented, and clearly presented work | Good documentation and presentation | Basic documentation with some gaps | Poor documentation and presentation |

### Marks Evaluation:

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Industry Problem | 20% | | | | |
| Application of Engineering Concepts | 25% | | | | |
| Solution Design | 25% | | | | |
| Use of Tools | 15% | | | | |
| Analysis | 10% | | | | |
| Professionalism | 5% | | | | |
| **Total Marks (100):** | | | | | |

| Faculty In-charge | Course Coordinator | Head of Department |
|---|---|---|
| Name: | Name: | Name: |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

**Smart Healthcare Resource Allocation (Optimization + Scheduling)**

**1. Problem Understanding**

Smart healthcare systems must efficiently allocate limited medical resources such as ICU beds, doctors, nurses, ventilators, and emergency treatment facilities among patients. In a multi-specialty hospital, the number of critical patients may exceed the available resources.

The objective is to allocate resources to patients in a way that maximizes patient care efficiency while giving higher priority to patients with more critical medical conditions.

Each patient can be assigned a priority value based on factors such as:

- Severity of the medical condition.
- Emergency level.
- Requirement for ICU treatment.
- Availability of hospital resources.
- Staff availability.
- Time required for treatment.

The system must ensure that the limited resources are allocated efficiently. Patients with higher priority should generally receive treatment before patients with lower priority.

The problem can be solved using an optimization and scheduling approach. A priority-based algorithm is used to sort patients according to their medical priority and allocate the available ICU beds to the most critical patients.

The algorithm should:

- Accept patient information and priority values.
- Sort patients according to priority.
- Allocate limited ICU beds.
- Identify patients who must wait.
- Calculate resource utilization.
- Display the final allocation schedule.

---

**Mathematical Formulation**

Let:

$P = \{p_1, p_2, p_3, ..., p_n\}$

represent the set of patients.

Let:

R = number of available ICU beds

Let:

$priority(p_i)$

represent the priority value of patient $p_i$.

Patients are sorted in descending order of priority:

$priority(p_1) \geq priority(p_2) \geq ... \geq priority(p_n)$

An ICU bed is allocated if:

$i \leq R$

Otherwise, the patient is placed in the waiting list.

The objective is to maximize the allocation of available medical resources while prioritizing critical patients.

---

**Input and Output Specification**

**Input**

The input consists of:

- A list of patient names.
- A priority value for each patient.
- The number of available ICU beds.

**Example**

Patients:

Patient A

Patient B

Patient C

Patient D

Patient E

Priority values:

5, 10, 8, 3, 7

Available ICU beds:

3

**Output**

The program displays:

Patient Allocation

Patient B (Priority: 10) - ICU Bed Allocated

Patient C (Priority: 8) - ICU Bed Allocated

Patient E (Priority: 7) - ICU Bed Allocated

Patient A (Priority: 5) - Waiting

Patient D (Priority: 3) - Waiting

It also displays resource utilization information.

---

**2. Constraint Analysis**

Constraint checking is important in healthcare resource allocation because hospital resources are limited.

The following constraints are considered.

**Patient Priority Constraint**

Patients with higher priority should be considered before patients with lower priority.

Condition:

$priority(p_i) > priority(p_j)$

This ensures that critical patients receive medical resources first.

**Resource Availability Constraint**

The number of allocated ICU beds cannot exceed the available number of beds.

Condition:

Allocated Beds ≤ Available Beds

This prevents over-allocation of hospital resources.

**Staff Availability Constraint**

A patient should only be assigned to treatment when the required medical staff is available.

For example:

Required Doctors ≤ Available Doctors

Required Nurses ≤ Available Nurses

This ensures that treatment schedules remain feasible.

**Emergency Arrival Constraint**

Emergency patients may arrive at any time.

A new patient with a very high priority may need immediate allocation.

The scheduling system should therefore allow priority updates.

**Completion Constraint**

The allocation process is complete when every patient has either:

- Received a medical resource allocation, or
- Been placed on the waiting list.

The most important constraints are patient priority and resource availability.

---

**3. State-Space Representation and Optimization Techniques**

The allocation process can be represented as a sequence of scheduling decisions.

Each patient represents a decision point.

For example:

Patient A → Priority 5

Patient B → Priority 10

Patient C → Priority 8

Patient D → Priority 3

Patient E → Priority 7

After sorting:

Patient B → 10

Patient C → 8

Patient E → 7

Patient A → 5

Patient D → 3

The algorithm allocates ICU beds starting from the highest-priority patient.

**Scheduling Decisions**

For each patient:

1. Check the patient's priority.
2. Check the availability of ICU beds.
3. Allocate a bed if resources are available.
4. Otherwise, place the patient on the waiting list.
5. Continue until all patients have been processed.

**Optimization Techniques**

The following techniques improve efficiency:

- Priority Sorting: Critical patients are processed first.
- Resource Constraint Checking: Prevents allocating more beds than available.
- Early Allocation: Stops unnecessary checks once all available beds are assigned.
- Dynamic Priority Handling: Allows emergency patients to receive immediate attention.
- Waiting List Management: Patients without available resources remain scheduled for future allocation.

These techniques improve the efficiency of healthcare resource utilization.

---

**4. Algorithm Using Priority-Based Scheduling**

**INPUT**

Patient list

Priority values

Number of available ICU beds

**OUTPUT**

ICU allocation schedule

Waiting list

Resource utilization

**Pseudocode**

```
FUNCTION allocateResources(patients, priorities, icuBeds):

    patientPriority = combine patients with priorities

    sort patientPriority in descending order

    FOR each patient in patientPriority:

        IF allocatedBeds < icuBeds:

            allocate ICU bed to patient

            allocatedBeds = allocatedBeds + 1
```

```
    ELSE:
        place patient in waiting list
    calculate resource utilization
    display allocation results
END FUNCTION
```

---

**Important Steps**

**Sorting Patients**

Patients are combined with their priority values.

Example:

(Patient A, 5)

(Patient B, 10)

(Patient C, 8)

The patients are then sorted in descending order.

**Resource Allocation**

The algorithm checks whether an ICU bed is available.

If available:

ICU Bed Allocated

Otherwise:

Waiting

**Resource Utilization**

Resource utilization can be calculated using:

Utilization = (Allocated Resources / Total Available Resources) × 100

This helps measure how efficiently the hospital resources are being used.

---

**5. Execution and Testing**

**Sample Input**

```
Patients = ["Patient A", "Patient B", "Patient C",
        "Patient D", "Patient E"]

Priorities = [5, 10, 8, 3, 7]

ICU Beds = 3
```

**Execution Trace**

**Step 1: Combine Patient and Priority**

Patient A → 5

Patient B → 10

Patient C → 8

Patient D → 3

Patient E → 7

**Step 2: Sort Patients by Priority**

After sorting:

Patient B → 10

Patient C → 8

Patient E → 7

Patient A → 5

Patient D → 3

**Step 3: Allocate First ICU Bed**

Patient B has the highest priority.

ICU Bed Allocated

Allocated Beds = 1

**Step 4: Allocate Second ICU Bed**

Patient C is the next highest-priority patient.

ICU Bed Allocated

Allocated Beds = 2

**Step 5: Allocate Third ICU Bed**

Patient E is next.

ICU Bed Allocated

Allocated Beds = 3

**Step 6: Remaining Patients**

No ICU beds are available.

Patient A → Waiting

Patient D → Waiting

---

**Output**

Patient Allocation

----------

Patient B (Priority: 10) - ICU Bed Allocated

Patient C (Priority: 8) - ICU Bed Allocated

Patient E (Priority: 7) - ICU Bed Allocated

Patient A (Priority: 5) - Waiting

Patient D (Priority: 3) - Waiting

---

**6. Correctness Justification**

The algorithm is correct because it processes all patients and allocates the available ICU beds according to patient priority.

At every step:

- Patients are sorted according to their priority values.
- Higher-priority patients are considered before lower-priority patients.
- The algorithm checks whether an ICU bed is available.
- A patient receives an ICU bed only when resources are available.
- Patients who cannot receive a bed are placed on the waiting list.

If the number of available ICU beds is R, the algorithm allocates beds to the first R patients in the priority-sorted list.

Therefore, no more than R patients receive ICU beds.

Since the patients are sorted in descending order of priority, the allocated patients have the highest priority among the available patients.

Thus, the algorithm provides a valid priority-based healthcare resource allocation.

---

**7. Complexity Analysis**

**Time Complexity**

Suppose:

n = number of patients

The major operation is sorting the patients according to priority.

Sorting requires:

O(n log n)

After sorting, the allocation process checks every patient once.

This requires:

O(n)

Therefore, the overall time complexity is:

O(n log n)

because the sorting operation dominates the execution time.

---

**Best-Case Complexity**

Even in the best case, patients must generally be sorted according to priority.

Therefore:

O(n log n)

---

**Space Complexity**

The algorithm stores:

- The patient list.
- Priority values.
- Patient-priority pairs.
- Allocation results.

Therefore, the auxiliary space requirement is approximately:

O(n)

---

**8. Performance Evaluation**

The performance of the healthcare resource allocation system can be evaluated using the following metrics.

**Response Time**

Response time measures how quickly the system makes allocation decisions.

A lower response time is important in emergency healthcare systems.

**Resource Utilization**

Resource utilization measures how effectively the available ICU beds are used.

Formula:

Utilization = (Allocated ICU Beds / Available ICU Beds) × 100

**Patient Priority Satisfaction**

This measures whether patients with higher medical priority receive resources before lower-priority patients.

**Waiting Time**

The waiting time of patients should be minimized.

Patients with lower priority may need to wait until resources become available.

**Scalability**

The system should efficiently handle a larger number of patients.

Priority-based scheduling is suitable for moderate-sized healthcare resource allocation systems.

---

**9. Advantages and Limitations**

**Advantages**

- Easy to understand and implement.
- Prioritizes critical patients.
- Efficiently uses available medical resources.
- Prevents resource over-allocation.
- Provides a clear waiting list.
- Sorting improves scheduling efficiency.
- Can be extended to include doctors, nurses, ventilators, and operation rooms.
- Suitable for emergency healthcare environments.

**Limitations**

- Simple priority values may not represent every medical situation.
- Emergency patient conditions can change dynamically.
- Real hospitals require multiple resource constraints.
- Staff schedules may make the problem more complex.
- Patients with similar priority values may require additional decision rules.
- A real-time system may require advanced optimization algorithms.

---

**Conclusion**

The Smart Healthcare Resource Allocation problem demonstrates how optimization and scheduling techniques can improve hospital resource utilization.

The priority-based scheduling approach allocates limited ICU beds to patients with the highest medical priority. Patients are sorted according to priority, and the available resources are assigned efficiently.

The algorithm ensures that:

- Critical patients are prioritized.
- ICU beds are not over-allocated.
- Lower-priority patients are placed on a waiting list.
- Resource utilization can be measured.

The approach is simple and efficient, with a time complexity of:

O(n log n)

For the given example, the system successfully allocates the three available ICU beds to the highest-priority patients:

Patient B

Patient C

Patient E

The remaining patients are placed on the waiting list.

Thus, priority-based optimization provides an effective approach for improving patient care efficiency and healthcare resource utilization.

OUTPUT